



Edge Impulse and OPENMV Mini Project

EE6008 Deep Learning at the Edge

Jocelyn Murphy, Abhishek Pandey, Saransh Bhatnagar,
Camille Conjat, and Abhishek Reddy Mudiya

Student ID's: 24246964, 24113867, 24244368, 24128759, 24202177

Abstract

This mini project explores the development of an image classification system for American Sign Language (ASL) using Edge Impulse and the OpenMV camera. The objective was to create a lightweight, real-time sign language recognition model deployable on edge devices. A custom dataset of ASL hand signs was collected using the OpenMV camera. The dataset was then used to train a convolutional neural network on the Edge Impulse platform. The trained model was deployed back to the OpenMV camera for on-device inference, enabling real-time classification of hand signs without the need for cloud processing. The results demonstrated promising accuracy in controlled environments, highlighting the potential of edge-based AI for accessible and portable communication tools. This project showcases how embedded machine learning can support assistive technologies with minimal hardware requirements.

Dataset

To train the ASL classification model, a custom image dataset was created using the OpenMV camera. Each of the five team members participated in the data collection process to ensure diversity in hand shapes, sizes, and orientations. For each letter of the ASL alphabet, excluding 'J' and 'Z', 10 static images were captured per person, resulting in 50 images per letter.

The letters 'J' and 'Z' were omitted from the dataset due to their reliance on motion, which cannot be effectively captured using single-frame image classification. Instead, representative static gestures were substituted to maintain the full 26-letter set: the letter 'J' was represented by a thumbs up and 'Z' was represented by an open palm. These substitutions simplified the classification task by using static images only.

The dataset captured a broad range of variations in lighting, background, and hand positioning. However, since the OpenMV camera has limited processing power and memory, all images needed to be resized to a lower resolution to ensure compatibility with both the training and the real-time inference. Images were resized to 96×96 pixels, which provided a balance between preserving visual detail for classification and meeting the resource constraints of the OpenMV hardware. This resizing process was completed through Edge Impulse.

Model Design

The model was created in Edge Impulse and trained using a convolutional neural network (CNN). The architecture for the final model can be seen in **Figure 1** below.

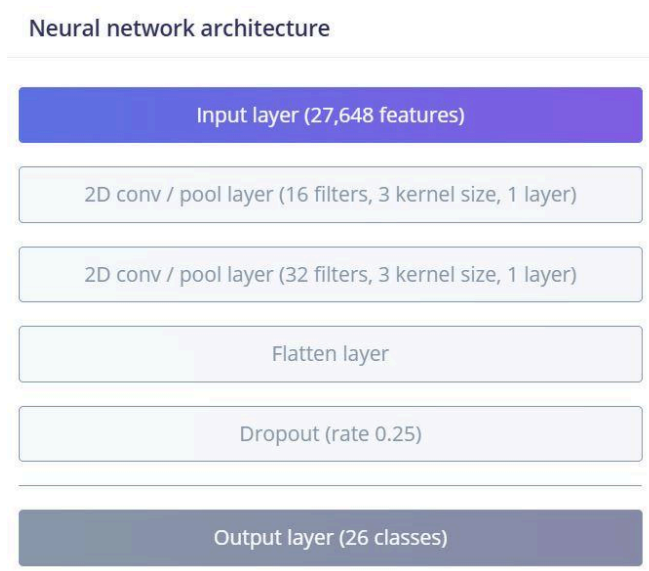


Figure 1: CNN Architecture from Edge Impulse

When trained with this architecture, the model reached 87.4% accuracy on the training set and 84.7% on the test set. The results of which can be viewed on the public Edge Impulse project [here](#) as well as in **Figure 2**.



Figure 2: Edge Impulse Model Results

After training the ASL classification model on Edge Impulse, the resulting TensorFlow Lite (TFLite) model was deployed on the OpenMV platform for inference. The pre-trained model file (trained.tflite) and its corresponding label file (labels.txt) were loaded into memory, with a built-in check to ensure the model size did not exceed the available RAM. For testing, the program was manually started and stopped on the OPENMV camera, which resulted in a static image that was then resized to 240×240 pixels in RGB format. The model then performed inference on the image using `net.predict`, generating a list of class probabilities. These results were printed alongside the corresponding ASL label and the inference time. The code (snapshot.py) used for OPENMV can be seen at the github link here: [GitHub - Deep Learning Project](#). Code from MIT is also available in GitHub as `ei_image_classification.py` as it was heavily referenced while building our model [1].

Results

The performance of the ASL image classification model on the OpenMV camera yielded mixed results. While some hand signs, such as the letter 'A', consistently achieved up to 96% accuracy during inference, many others exhibited significantly lower and inconsistent performance. For example, letters with distinct hand shapes like 'V' or 'K' produced variable inference results, with confidence levels ranging from 9% to 61% depending on lighting and background conditions.

A common issue observed during testing was confusion between similar-looking gestures, where the model assigned meaningful confidence percentages to multiple letters at once. This suggests that the model struggled to clearly differentiate between hand signs that share similar visual features, especially when captured from slightly varied angles or positions.

Testing was noticeably more reliable in environments that closely resembled those used during dataset collection, such as team members' homes and the library. However, when testing in new environments, the model displayed sensitivity to changes in lighting and background texture, which indicates that the model had limited generalization ability. This was likely due to the relatively small dataset size and lack of diversity in environment and lighting.

Despite the challenges, the project demonstrated the potential for using embedded machine learning on low-power devices like the OpenMV camera. These results underscore the importance of larger, more diverse datasets and further model tuning to improve real-world robustness.

Challenges and Reflections

This project involved a series of experiments and technical challenges that ultimately shaped the final outcome. Initially, we explored using the MNIST American Sign Language dataset, training it through the Edge Impulse platform [2]. However, the dataset's large size overwhelmed the platform, leading to limited success in both training and testing accuracy. Hoping for better results, we shifted to training the MNIST dataset using Keras with TensorFlow and quantized the model to INT8 format for compatibility with OpenMV. Unfortunately, deploying this quantized TFLite model led to runtime errors on the camera, which we were unable to resolve. The coding for training the MNIST model can be found [here](#) as well as the quantization of the model, [here](#).

When debugging the MNIST Keras with Tensorflow model was unsuccessful, we decided to build and train our own custom dataset using the OpenMV camera. While this allowed for a streamlined workflow between data collection, training, and deployment, it also introduced several new challenges. One of the most significant issues was the lack of consistency in our dataset. Team members captured images in various environments with different lighting and background conditions. Additionally, some team members used their left hand while others used their right, an inconsistency that was only noticed later in the process. These variations reduced the model's ability to generalize, as it struggled to distinguish between letters when presented with unfamiliar conditions during testing. While this variety in the images would be ideal for a

much larger dataset, the small size of the dataset made these differences difficult in terms of generalization.

Another difficulty was related to the number of output classes. Training a model with 26 different letter classes on Edge Impulse proved problematic as the model often failed to confidently recognize specific gestures. Even though some letters achieved high confidence during inference, others resulted in split confidence scores across multiple labels. This issue may have been affected by the need to trim the dataset to meet platform constraints. Although the model reported high training accuracy, its real-world performance in varied environments was often poor. Image size posed another challenge; while smaller resolutions allowed faster processing, they also reduced the model's ability to extract meaningful features from the input.

Despite these setbacks, the project provided a valuable learning experience. It underscored the importance of data consistency, thoughtful class design, and the limitations inherent to deploying machine learning models on embedded hardware. These insights will guide our future efforts to build more robust and reliable edge AI systems.

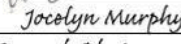
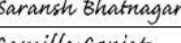
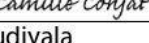
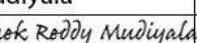
If given more time, we would focus on several key improvements to enhance the accuracy and reliability of our model. First, we would slow down the frame rate on the OpenMV camera to reduce the processor strain and allow a more stable inference per frame. Additionally, we would revisit our Keras/TensorFlow quantized model by first importing it into Edge Impulse to take advantage of its optimization tools before deploying to OpenMV. Most importantly, we would rebuild our dataset with strict consistency and ensure all images are captured against a clean background, using the same hand, and under uniform lighting conditions. These changes would help improve model generalization and real-time performance.

Resources

[1] "Collecting image data with the OpenMV Cam H7 Plus | Edge Impulse Documentation," *Edgeimpulse.com*, Feb. 06, 2025. Available: <https://docs.edgeimpulse.com/docs/tutorials/data/data-ingestion/image-classification-openmv>. [Accessed: May 02, 2025]

[2] "Sign Language MNIST," *www.kaggle.com*. Available: <https://www.kaggle.com/datasets/datamunge/sign-language-mnist>

Appendix A: Team Member Contribution Statement

Name	Signature	Key contributions	Estimated % contribution
Abhishk Pandey		Dataset, coding, testing, presentation	20
Jocelyn Murphy		Dataset, coding, testing, documentation	20
Saransh Bhatnagar		Dataset, coding, testing, presentation	20
Camille Conjat		Dataset, testing, project planning	20
Abhishek Reddy Mudiya		Dataset, testing, project planning	20
	